front matter

foreword

Well-grounded? You mean, "well-rounded"? Two years of pandemic would do that without the need for a book.

Merriam-Webster defines well-grounded as "having a firm foundation." I like that. We want to have a firm foundation in Java—a practical knowledge of what we need to know to call ourselves Java experts. This book picks up where Effective Java stops.

This is the second edition of a great book. The first taught us all that we needed to know for Java 7. That seems like eons ago. Java 7 belonged to another age, when features were added to the language at best every three years. Back then, it was easy to keep versions apart. Java 5? Generics and enums. Java 7? `try-with-resource`. Java 8? Streams and lambdas. Those comfortable easy days ended when Oracle introduced the six-month cycle. Records—were those Java 14, 15, or 17? Enhanced Switch? Was that already in Java 11?

The fast release cycle is great for programmers who work for adventurous companies. Every six months, they get new toys to play with. They might even get to try out previews of what will come next. The myriad of new features is wonderful for programmers, but not so nice for authors. Before the ink has dried, a new feature release makes a bunch of things obsolete.

Ben, Jason, and Martijn have done a fantastic job with this new Java book. The basic premise remains the same. In my words: "If you wanted to hire a professional Java programmer, what would you expect them to already know? What skills would they need to prove they are well grounded?"

This new version of the book is as current as is possible with the six-month release cadence. At the same time, the authors don't overwhelm us with new stuff. The stark reality is that most enterprises are still stuck on an older version of Java. Even with Java 18 released, a lot of banks, insurance companies, and government departments are still on Java 8.

This book is about 200 pages longer than the previous edition. The fonts are a bit larger—well, we have all aged by nine years, haven't we? But the margins are smaller. Quite a few of the sections have completely new content. This is one case where the new edition does not make the old one obsolete. Both belong on a serious Java programmer's bookshelf.

Benjamin J. Evans, Jason R. Clark, and Martijn Verburg are Java experts. They hold senior Java positions at Red Hat, New Relic, and Microsoft. Let's take advantage of their collective wisdom. This book will help us discover areas of weakness that we can then improve on. In the end, with enough work, we can call ourselves *Well-Grounded Java programmers*.

HEINZ KABUTZ

THE JAVA SPECIALISTS' NEWSLETTER

preface

The first edition of this book started life as a set of training notes written for new graduate developers in the foreign exchange department of a bank. One of us (Ben), looking at the existing books on the market, found a lack of up-to-date material aimed at Java developers who wanted to level up. Partway through writing that material, he found he was writing that missing book and enlisted Martijn to help.

That was more than 10 years ago—we were writing as Java *7* was being developed—and the world is very different now. In response, the book has changed substantially since the first edition. So, although our original primary goals were to introduce topics like

- Polyglot programming
- Dependency injection
- Multithreaded programming
- Sound build and CI practices
- What's new in Java 7

when we came to write the second edition, we found that we needed to make some changes, including

- Trimming down polyglot a bit
- Adding a new emphasis on functional programming
- Enhancing the discussion of multithreading
- Putting a different spin on build and deploy (including containers)

- Talking about what's new in Java 11 and 17

One very important change was that the first edition included Scala as one of three non-Java languages discussed (the others being Groovy and Clojure—Kotlin didn't really exist at the time we wrote it). At that time, many of the developers exploring Scala were looking for "Java, but a better mousetrap," which is essentially the view of Scala that we presented in the first edition.

However, since then, the world has moved on. Java 8 and 11 became hugely dominant, and the "better mousetrap" crew are mostly writing Kotlin (or just sticking with Java). Scala, in the meantime, has become a very powerful statically typed, FP-first JVM language. This is great for the folks who want that, but it has come with costs, such as an ever-more-complex runtime and a language that has less and less in common with Java as time goes by.

This development is sometimes abbreviated to the phrase that "Scala wants to be Haskell on the JVM," although this is not entirely accurate and is more a convenient conversational shorthand than anything else. So, after having made the decision to drop Groovy from the second edition, we thought long and hard about whether to keep Scala or replace it with Kotlin.

Our eventual conclusion was basically that Scala is heading in its own, FP-heavy direction, and that we wanted to present a language that was more approachable to Java developers who were coming fresh to non-Java languages, such as Kotlin. This left us with a dilemma. The parts of Scala that are easily accessible to Java folks are very similar to Kotlin (with near-identical syntax in some cases), but the philosophy and direction of travel of the two languages are totally different. We felt that explaining what Scala *is* in sufficient depth—so that the coverage was distinct from Kotlin—would take up far too much space in the book.

Therefore, our eventual decision was to drop from three to two additional languages to make space and give extra depth to the coverage of the remaining languages (Kotlin and Clojure). For this reason, although we make the occasional comment about Scala, we don't devote entire sections (let alone chapters) to it.

Clojure is a very different story—and, indeed, a very different language—than either Kotlin or Java. For example, in chapter 15 we struggle a bit because many of the concepts that we're introducing in the other languages (e.g., higher-order functions and recursion) have already been introduced and are just "part of the landscape" in Clojure. Rather than follow the template used by Java and Kotlin, the discussion goes in a differ-

ent direction. Clojure is, fundamentally, a much more functionally oriented language, and we would just end up repeating ourselves a lot if we were to follow the exact same structure as for the other languages.

In this book, we hope that the theme of software development as a social activity rings out clearly. We believe that the technical aspects of the craft are important, but the more subtle concerns of communication and interaction between people are at least as important. It can be hard to explain these facets easily in a book, but that theme is present throughout.

Developers are sustained throughout their careers by their engagement with technology and the passion to keep learning. In this book, we hope that we've been able to highlight some of the topics that will ignite that passion. It's a sightseeing tour, rather than an encyclopedic study, but that's the intention—to get you started and then leave you to follow up on those topics that capture your imagination.

We take you from the new features of the recent versions of Java through to best practices of modern software development and the future of the platform. Along the way, we show you some of the highlights that have had relevance to us on our own journey as Java technologists.

Concurrency, performance, bytecode, and class loading are some of the core techniques that fascinated us the most. We also talk about new, non-Java languages on the JVM for two reasons:

- Non-Java languages continue to gain importance in the overall Java ecosystem.
- Understanding the different perspectives that different languages bring makes you a better programmer in any language you write in.

Above all, this is a journey that's forward looking and puts you and your interests front and center. We feel that becoming a well-grounded Java developer will help to keep you engaged and in control of your own development and will help you learn more about the changing world of Java and the ecosystem that surrounds it. We hope that the distilled experience that you're holding in your hands is useful and interesting to you and that reading it is thought provoking and fun. Writing it certainly was!

acknowledgements

We would like to thank the following people for their contributions to the book:

Elesha Hyde, for being a most excellent development editor; Jonathon Thoms, for his great work in the technical reviews; Alex Buckley, for a very detailed discussion of the class loading process; Heinz Kabutz, for excellent suggestions and discussions (and even PRs!) about the details of the concurrency chapters, as well as another wonderful foreword; Holly Cummins, not only for helping inspire the original edition but also for her consistently grounded and practical advice; Bruce Durling, for discussion of the Clojure material; Dan Heidinga, for detailed feedback on the current state of Project Valhalla; Piotr Jagielski, Louis Jacomet, József Bartók, and Tom Tresansky for corrections regarding some of the details about how Gradle really works; and Andrew Binstock, for a very meticulous close reading of several chapters and sound advice, as always.

We would also like to thank the staff at Manning: Mihaela Batinić, our reviewing editor; Michael Haller, our technical reviewer; Deirdre Hiam, our project editor; Pamela Hunt, our copyeditor; and Jason Everett, our proofreader. To all the reviewers: Adam Koch, Alain Lompo, Alex Gout, Andres Sacco, Andy Keffalas, Anshuman Purohit, Ashley Eatly, Christian Thoudahl, Christopher Kardell, Claudia Maderthaner, Conor Redmond, Dr. Irfan Ullah, Eddú Meléndez Gonzales, Ezra Simeloff, George Thomas, Gilberto Taccari, Hugo da Silva Possani, Igor Karp, Jared Duncan, Javid Asgarov, Jean-François Morin, Jerome Meyer, Kent R. Spillner, Kimberly L Winston-Jackson, Konstantin Eremin, Matt Deimel, Michael Haller, Michael Wall, Mikhail Kovalev, Patricia Gee, Ramanan Natarajan, Raphael Villela, Satej Kumar Sahu, Sergio Edgar Martínez Pacheco, Simona Ruso, Steven K. Makunzva, Theofanis Despoudis, Troi Eisler, Yogesh Shetty, and William E. Wheeler, your suggestions helped make this a better book.

rom Jason

Thanks are due to many different people across many years.

To Mrs. Nimmo, thank you for all the extra credit in middle school English for my silly little stories. It's no exaggeration that your encouragement set me on a lifelong path of writing.

To Mom, thank you for your infectious love of reading, which I'm so glad to have inherited.

To Dad, thank you for sharing your love of computers. It has provided me not only with a career but the chance to write and share that joy with others.

To Ben, thank you first and foremost for your friendship. It has been a blast to be drawn deeper into the JVM by your awesome curiosity and enthusiasm. And, of course, thanks

for asking me along on this second edition. It was more work than any of us expected, but a better book in the end as well!

And last but not least, thanks to my wife, Amber, and my kids, Coraline and Asher, for their continued love and support throughout the strange and wonderful process that is making a book.

rom Martijn

Firstly I'd like to thank Ben and Jason for inviting me to be part of this second edition. My contribution was *very* minimal in comparison to theirs, and they were most gracious in insisting my name still be on the cover!

To Kerry, you've been a mountain of support during the whirlwind career moments of the past decade, not to mention reacting with a smile when I said, "It's just a few edits this time around, I promise!"

To Hunter, your enthusiasm for life reminds me why I got into the creative joy of programming in the first place. I hope you'll find that same joy in life no matter what path you take.

To the fine folks of the Java Engineering Group at Microsoft, the Eclipse Adoptium Community, the London Java Community, the Java Champions Community, and too many others to mention. You inspire me every day, and I always walk away each day having learned something new and added another five things on my list to read for the next day!

rom Ben

To my parents, Sue and Martin, for their unwavering faith that we would find, and make, our own way on the path less traveled.

To my wife, Anna, for her illustrations, her artistic vision, and her tireless support and understanding through yet another book.

In memory of Marianito, who, partway through the development of this book, discovered that laptops that have been left open make a marvelously warm spot upon which to sleep.

To Joselito, who overcame some of his fear by being curious about why I would sit and be so fascinated by the screen that was so much less interesting than the one in the

other room that has spaceships and explosions on it.

about this book

Who should read this book?

Welcome to *The Well-Grounded Java Developer.* This book is aimed at turning you into a Java developer for the next decade, reigniting your passion for both the language and platform. Along the way, you'll discover new Java features, ensure that you're familiar with essential modern software techniques (such as test-driven development, and container-based deployments), and start to explore the world of non-Java languages on the JVM.

To begin, let's consider the description of the Java language provided by James Iry in a wonderful blog post, "A Brief, Incomplete, and Mostly Wrong History of Programming Languages" available at **http://mng.bz/2rz9**:

1996—James Gosling invents Java. Java is a relatively verbose, garbage collected, class-based, statically typed, single dispatch, object-oriented language with single implementation inheritance and multiple interface inheritance. Sun loudly heralds Java's novelty.

Although the point of Java's entry is mostly to set up a gag where C# is given the same writeup, this is not bad as descriptions of languages go. The full blog post contains a bunch of other gems, and it's well worth a read in an idle moment.

This does present a very real question: why are we still talking about a language that is now over 26 years old? Surely it's stable, and not much new or interesting can be said about it?

If that were the case, this would be a short book. We are still talking about it because one of Java's greatest strengths has been its ability to build on a few core design decisions, which have proven to be very successful in the marketplace:

- Automatic management of the runtime environment (e.g., garbage collection, just-in-time compilation)
- A simple syntax and relatively few concepts in the core language
- A conservative approach to evolving the language
- Additional functionality and complexity in libraries
- Broad, open ecosystem

These design decisions have kept innovation moving in the Java world—the simple core has kept the barrier to joining the developer community low, and the broad ecosystem has made it easy for newcomers to find pre-existing components that fit their needs. These traits have kept the Java platform and language strong and vibrant—even if the language has had a historical tendency to change slowly. It turns out that the mix of strong consistency and evolutionary change has won quite a few fans among software developers.

ow to use this book

The material in this book is broadly designed to be read end-to-end, but we understand that some readers may want to dive straight into particular topics, so we have done our best to also accommodate that style of reading.

We strongly believe in hands-on learning, so we recommend that readers try out the sample code that comes with the book as they read through the text. The rest of this section deals with how you can approach the book if you are more of a standalone chapter style of reader.

The Well-Grounded Java Developer is split into the following five parts:

- From 8 to 11, and beyond
- Under the hood
- Non-Java languages on the JVM
- Build and deployment
- Java frontiers

Part 1 (chapters 1–3) contains three chapters on the most recent versions of Java. The book as a whole uses Java 11 syntax and semantics throughout and calls out specific uses of post-11 syntax.

Part 2 (chapters 4–7) contains a first peek behind the curtain. It is a truism of art that one needs to know the rules before one can credibly break them. These chapters outline how one first bends, and then breaks, the rules of the Java programming language.

Part 3 (chapters 8–10) covers polyglot programming on the JVM. Chapter 8 should be considered required reading because it sets the stage by discussing the categorization and use of alternative languages on the JVM.

The following two language chapters cover a Java-like OO-functional language (Kotlin) and a truly functional one (Clojure). Those languages can be read standalone, although

developers new to functional programming will probably want to read them in order.

Part 4 (chapters 11–14) introduces build, deployment, and testing as they are done in modern projects, and they assume that the reader has at least a basic understanding of unit testing as showcased in, for example, JUnit.

Part 5 (chapters 15–18) builds on topics that have been introduced earlier to delve deeper into functional programming, concurrency, and the internals of the platform. Although the chapters can be read standalone, in some sections, we assume that you've read the earlier chapters and/or already have familiarity with certain topics.

This book is firmly aimed at Java developers who wants to modernize their knowledge base in both the language and the platform. If you want to get up to speed with what modern Java has to offer, this is the book for you.

If you are looking to brush up on your techniques and understanding of topics such as functional programming, concurrency, and advanced testing, this book will give you a good grounding in those topics. This is also a book for those developers who are curious about what non-Java languages can teach them and how broadening their horizons will make them a better programmer.

bout the code

The initial download and installation you'll need is Java 17 (or 11). Simply follow the download and installation instructions for the binary you need for the OS you use. You can find binaries and instructions online at your usual Java vendor, or at the vendor-neutral Adoptium project, run by the Eclipse Foundation, at **https://adoptium.net/**.

Java 11 (and 17) runs on Mac, Windows, Linux, and pretty much any other modern OS and hardware platform.

NOTE If you're concerned about details of Java licensing and so on, you can head to appendix A where a full discussion can be found.

This book contains many examples of source code both in numbered listings and in line with normal text. In both cases, source code is formatted in a `fixed-width font like this` to separate it from ordinary text.

In many cases, the original source code has been reformatted; we've added line breaks and reworked indentation to accommodate the available page space in the book. Additionally, comments in the source code have often been removed from the listings

when the code is described in the text. Code annotations accompany many of the listings, highlighting important concepts.

You can get executable snippets of code from the liveBook (online) version of this book at **https://livebook.manning.com/book/the-well-grounded-java-developer-second-edition**. The complete code for the examples in the book is available for download from the Manning website at **https://www.manning.com/books/the-well-grounded-java-developer-second-edition**, and from GitHub at **https://github.com/well-grounded-java/resources**.

However, most readers will probably want to try out the code samples in an IDE. Java 11/17 and the latest versions of Kotlin and Clojure are well supported by recent versions of the main IDEs:

- Eclipse IDE
- IntelliJ IDEA Community Edition (or Ultimate Edition)
- Apache NetBeans

veBook Discussion Forum

Purchase of *The Well-Grounded Java Developer, 2nd Edition* includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to **https://livebook.manning.com/book/the-well-grounded-java-developer-second-edition/discussion**. You can also learn more about Manning's forums and the rules of conduct at **https://livebook.manning.com/discussion**.

Manning's commitment to our readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the authors can take place. It is not a commitment to any specific amount of participation on the part of the authors, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking the authors some challenging questions lest their interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.

ther online resources

https://github.com/well-grounded-java/resources

about the authors



Ben Evans is a Java Champion and Senior Principal Software Engineer at Red Hat. Previously he was Lead Architect for Instrumentation at New Relic, and co-founded jClarity, a performance tools startup acquired by Microsoft. He has also worked as Chief Architect for Listed Derivatives at Deutsche Bank and as Senior Technical Instructor for Morgan Stanley. He served for six years on the Java Community Process Executive Committee, helping define new Java standards.

Ben is the author of six books, including Optimizing Java and the new editions of Java in a Nutshell and his technical articles are read by thousands of developers every month. Ben is a regular speaker and educator on topics such as the Java platform, systems architecture, performance and concurrency for companies and conferences all over the world.



Jason Clark is a principal engineer and architect at New Relic where he has worked on everything from Ruby instrumentation libraries to container orchestration platforms. He was previously an architect at WebMD building .Net-based web services.

A regular conference speaker, Jason contributes to the open-source project Shoes, aiming to make GUI programming easy and fun for beginners and students.



Martijn Verburg is the Principal SWE Group Manager for the Java Engineering Group at Microsoft. He is the co-leader of the London Java User Group (aka the LJC) where he co-founded AdoptOpenJDK (now Eclipse Adoptium), the world's leading (non- Oracle) OpenJDK distribution. Martijn is the co-author of *The Well-Grounded Java Developer, 1st edition*, and sits on numerous Java standards bodies (JCP, Jakarta EE, et al).

about the cover illustration

The figure on the cover of *The Well-Grounded Java Developer*, titled "A Posy Seller," is taken from Sylvain Maréchal's nineteenth-century compendium of regional dress customs, published in France.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Previous chapter
‹ contents

Next chapter
Part 1. From 8 to 11 and beyond! ›